



Introducción a la Ingeniería Web

Mtr. Jisbaj Gamarra Salas

¿Qué es la Ingeniería Web?

Disciplina que estudia **métodos, procesos, técnicas, herramientas y estándares** para construir sistemas web fiables, seguros, mantenibles y escalables. Aborda el **ciclo de vida completo**: desde el análisis de requisitos hasta la evolución y mantenimiento de la aplicación.

Metodologías y procesos

Enfoques iterativos, centrados en el usuario y orientados a contenido y servicios, específicos para proyectos web.

Aspectos técnicos

Arquitectura web, protocolos HTTP/HTTPS, lenguajes (HTML, CSS, JS), seguridad, rendimiento y accesibilidad.

Experiencia de usuario

Hipermedia, gestión de contenidos, integración empresarial y servicios en la nube.

- 📌 **Relevancia profesional:** La Web es la plataforma dominante en banca, educación, gobierno y comercio. La complejidad actual exige ingeniería rigurosa, más allá del desarrollo artesanal.

Ingeniería Web vs. Desarrollo Web vs. Ingeniería de Software

Desarrollo Web

Foco en la programación práctica: maquetación, scripting e integración de tecnologías concretas. No incluye sistemáticamente gestión de requisitos, modelado ni control de calidad.



Ingeniería Web

Añade procesos sistemáticos, gestión de requisitos, modelado, pruebas rigurosas, mantenimiento y gestión de riesgos. Adaptada a la naturaleza abierta, distribuida y orientada a contenidos de la Web.



Ingeniería de Software

Comparte principios (ciclo de vida, pruebas, diseño arquitectónico, métricas). La Ingeniería Web reutiliza estos principios, adaptándolos a alta interactividad, usuarios masivos y ciclos de despliegue muy cortos.

La ingeniería web **no es un subconjunto trivial** de la ingeniería de software; reutiliza sus principios, pero los adapta a la naturaleza abierta, distribuida y orientada a contenidos de la Web.

Roles y Panorama de la Industria

Roles típicos en proyectos web

Front-end

Arquitectura de interfaz, rendimiento en navegador y experiencia de usuario.

Back-end

Diseño de APIs, lógica de negocio, persistencia y despliegue en servidores.

Full-stack

Cobertura extremo a extremo, desde interfaz hasta infraestructura.

Arquitecto/a de soluciones

Define la arquitectura global: capas, servicios e integración con otros sistemas.

Especialistas

DevOps, seguridad web, accesibilidad, performance y datos.

Tendencias de la industria

El mercado demanda perfiles con **bases sólidas en protocolos, arquitecturas distribuidas, seguridad y prácticas de ingeniería**. La tendencia se orienta hacia:

- Arquitecturas basadas en servicios REST/JSON y GraphQL
- Microservicios y despliegue en la nube
- Comprensión profunda del ecosistema web

Ideas clave

1

Una disciplina de ingeniería

La ingeniería web no consiste solo en "programar páginas": aplica procesos sistemáticos, métricas de calidad y gestión del ciclo de vida completo.

2

Retos propios de la Web

Se apoya en la ingeniería de software, pero responde a desafíos únicos: hipertexto, escala masiva, usuarios heterogéneos y cambios continuos.

3

Dominio del ecosistema técnico

Comprender protocolos, navegadores y servidores es requisito indispensable para diseñar soluciones robustas y escalables.

Arquitectura de Aplicaciones Web

El ecosistema Web funciona gracias a un modelo **cliente-servidor** donde el navegador envía solicitudes HTTP a un servidor, que procesa la petición y devuelve una respuesta. Este mecanismo es la base de toda arquitectura web moderna.



Flujo Request / Response

Componentes del modelo

Cliente

Agente de usuario (navegador, app móvil, cliente HTTP) que inicia la comunicación.

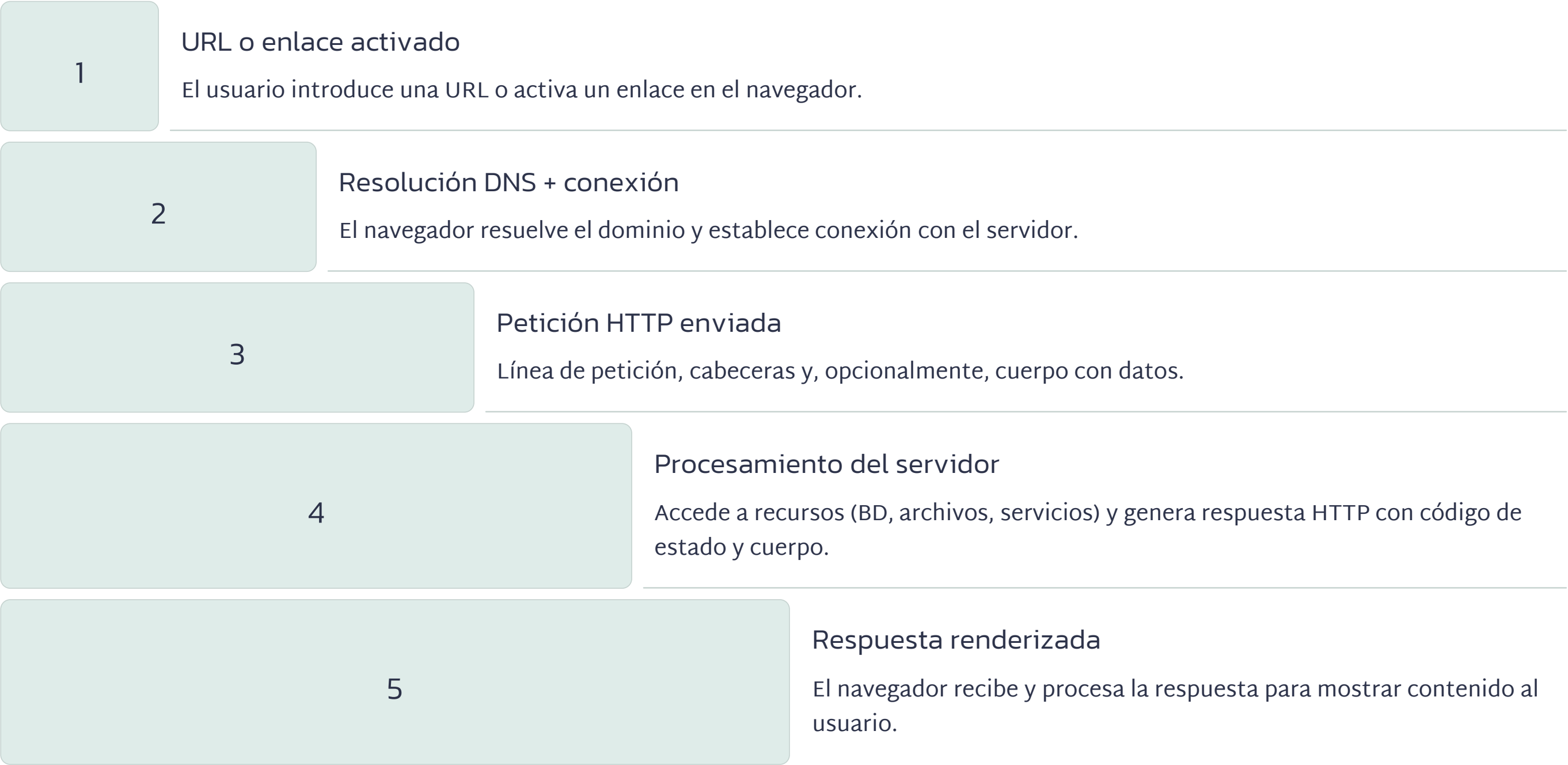
Servidor web / app

Recibe la petición, la interpreta y ejecuta lógica de aplicación para generar una respuesta.

HTTP/HTTPS

Define formato de mensajes, métodos (GET, POST...), códigos de estado y cabeceras.

Secuencia del ciclo



Piezas del Ecosistema Web

Una única interacción del usuario puede involucrar resolución DNS, enrutamiento por Internet, CDN, servidor web, lógica de aplicación y base de datos, todo coordinado a través de HTTP/HTTPS.



DNS y URL

DNS traduce nombres de dominio a IPs. La URL identifica esquema, dominio, puerto, ruta y parámetros de consulta.



Servidor Web y App

Nginx/Apache gestiona conexiones HTTP y sirve estáticos. El servidor de app ejecuta lógica de negocio (Django, Spring, Node.js).



Base de Datos

Persiste datos estructurados o semiestructurados accesibles desde la lógica de la aplicación (SQL o NoSQL).



CDN (Content Delivery Network)

Red distribuida de cachés que acerca contenidos estáticos a los usuarios para reducir latencia y mejorar rendimiento.

La pila tecnológica base de la Web



HTTP / HTTPS

Protocolo de aplicación para transferencia de recursos. HTTPS añade TLS/SSL para confidencialidad, integridad y autenticación.



HTML y CSS

HTML estructura el contenido semántico (títulos, párrafos, formularios). CSS describe la presentación visual (colores, tipografías, layouts responsivos) de forma independiente.



JavaScript

Lenguaje interpretado en el navegador que añade comportamiento dinámico, manipula el DOM y consume servicios remotos mediante APIs.



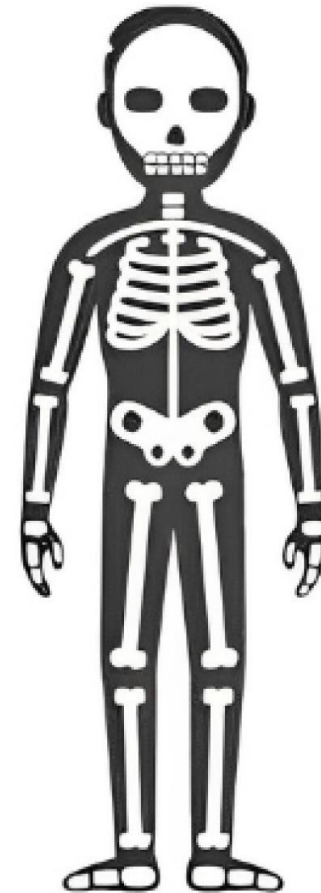
JSON y APIs Web

JSON es el formato ligero de intercambio de datos más extendido. Las APIs REST exponen recursos a través de HTTP, devolviendo típicamente JSON.

HTML



HTML the Skeleton



CSS



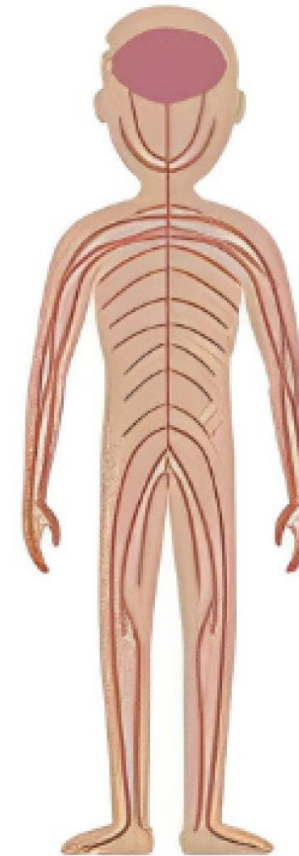
CSS the Skin



JS



Javascript the Brain



Ideas clave

El corazón de la Web

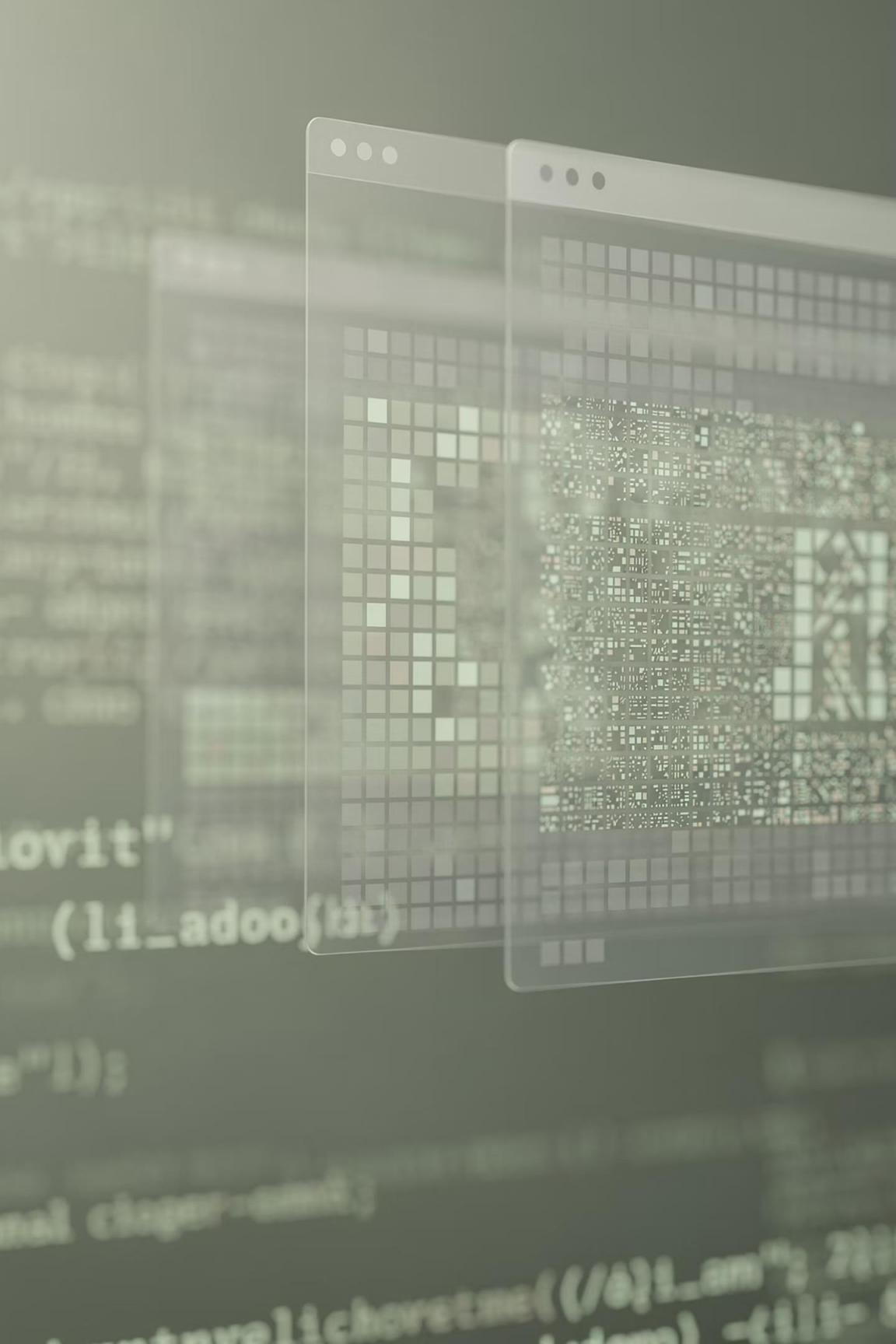
El modelo cliente–servidor y el ciclo HTTP request/response son el fundamento conceptual sobre el que se construye cualquier arquitectura web.

Un ecosistema distribuido

DNS, URL, servidores, bases de datos y CDNs cooperan de forma coordinada para entregar contenido al usuario final en milisegundos.

La pila mínima indispensable

HTTP/HTTPS, HTML, CSS, JavaScript, JSON y APIs son los "ladrillos" que cualquier ingeniero/a web debe dominar con rigor.



Navegadores: ¿Qué Hay Dentro y Qué Hacen?

Un navegador moderno es un entorno de ejecución extremadamente complejo, con motores especializados para renderizado y JavaScript, además de componentes para red, almacenamiento y gestión de procesos.

Motor de renderizado

Blink, WebKit, Gecko
— interpreta HTML/CSS y produce la representación visual.

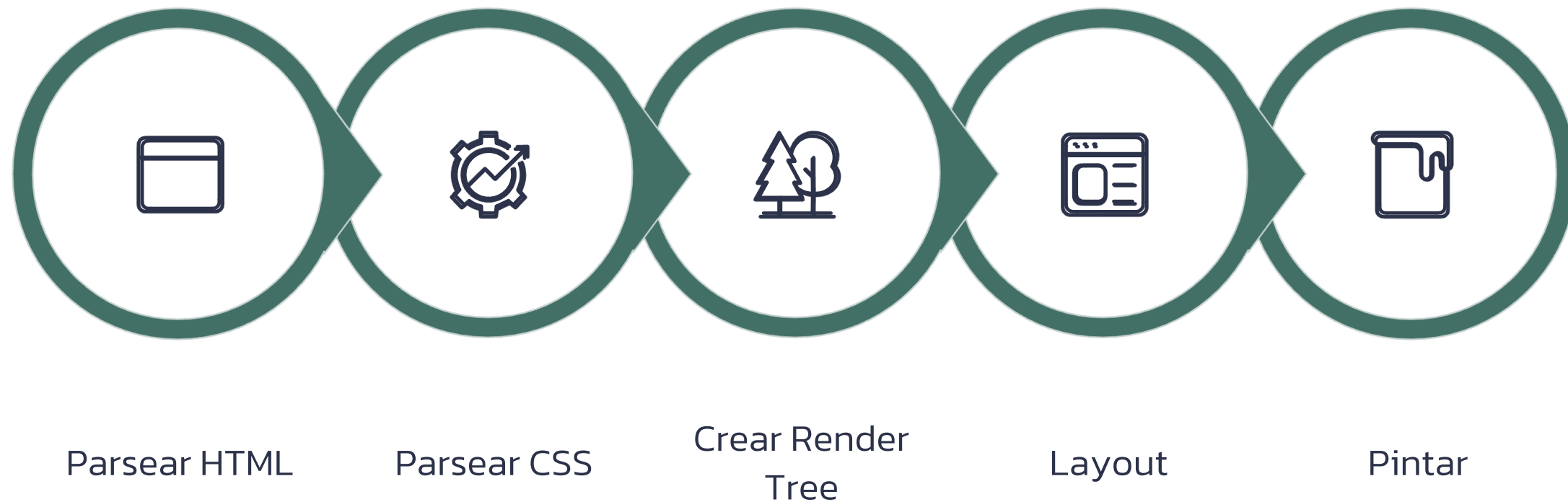
Motor JavaScript

V8, SpiderMonkey, JavaScriptCore — ejecutan código JS de forma optimizada.

Capa de red y almacenamiento

Gestiona peticiones HTTP, caché, cookies y Web Storage.

Pipeline de Renderizado: Del Código a los Píxeles



DOM y CSSOM

El **DOM** es la representación en árbol del documento HTML, donde cada elemento se convierte en un nodo jerárquico. El **CSSOM** representa las reglas CSS parseadas y su aplicación potencial sobre el DOM.

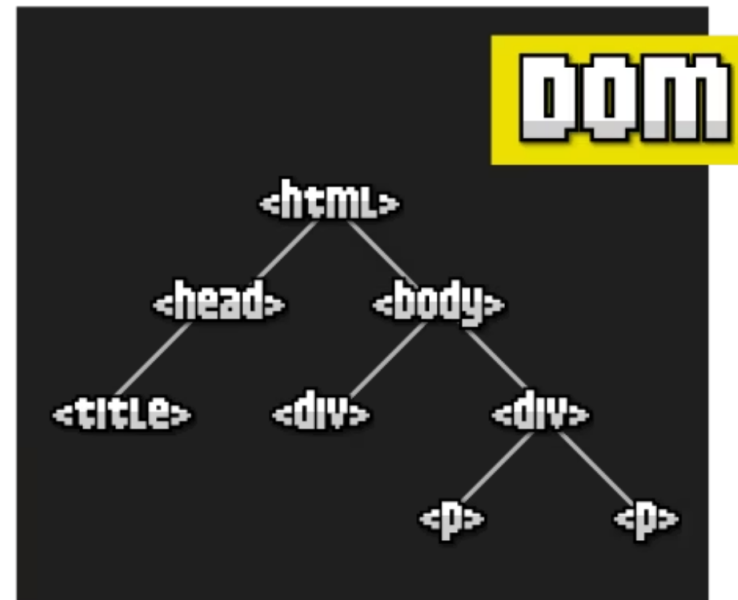
Layout y Paint

El **layout** calcula la geometría (posición y tamaño) de cada nodo visible. El **paint** traduce ese resultado en operaciones de dibujo para rasterizar píxeles en pantalla. Minimizar cambios que fueren estas fases es clave para el rendimiento.

El DOM (Document Object Model o Modelo de Objeto de Documento)

Es la lista estandarizada de elementos de una página web que pueden ser consultados y manipulados mediante lenguajes de programación, principalmente JavaScript . Es la interfaz que permite que un documento HTML estático se convierta en una experiencia interactiva y dinámica .

```
<html>
<head>
  <title>Title</title>
</head>
<body>
  <div></div>
  <div>
    <p>Párrafo 1</p>
    <p>Párrafo 2</p>
  </div>
</body>
</html>
```



1. La Metáfora del Árbol Técnicamente, el DOM representa la jerarquía de un documento como un árbol invertido que se ramifica desde una raíz .
 - a. La Raíz: Es el elemento `<html>` .
 - b. **Ramas y Hojas:** Son los elementos hijos como `<head>` y `<body>`, que a su vez contienen otros elementos como `<h1>`, `<p>`, `<img>`, etc.
 - c. **Relaciones:** En esta estructura se utilizan términos familiares: un elemento que contiene a otro es su **padre** (*parent*), y el elemento contenido es su **hijo** (*child*)

EL DOM

- Cada parte del documento se considera un nodo . No solo las etiquetas son parte de este árbol; nodos de elemento, nodos de texto, nodos de atributo: La información extra de las etiquetas (como el src de una imagen o el href de un enlace) . Incluso los comentarios que dejas en el código son parte del DOM .
- ¿Para qué sirve? El DOM es la herramienta que usa JavaScript para "darle órdenes" al navegador y cambiar la página en tiempo real sin necesidad de recargarla . Gracias al DOM, JavaScript puede: Acceder a elementos:
 1. Por ejemplo, buscar un párrafo específico usando `document.querySelector("p")` .
 2. Modificar contenido: Cambiar el texto de un título o el HTML interno de una sección .
 3. Alterar atributos: Cambiar la dirección de un enlace o la fuente de una imagen .
 4. Crear y eliminar: Generar nuevos botones o párrafos desde cero y añadirlos a la página, o borrar elementos existentes .
- El punto de entrada principal para trabajar con el DOM es el objeto **document** . Este objeto representa la página HTML completa cargada en el navegador y contiene todas las propiedades y métodos necesarios para navegar y modificar el árbol de nodos

Event Loop, Web APIs y Almacenamiento en el Navegador

Modelo de ejecución JavaScript

JS se ejecuta en un **único hilo principal** con un modelo basado en **event loop**: gestiona una cola de tareas (clics, timers, respuestas de red) y ejecuta el código asociado de forma secuencial, evitando condiciones de carrera.

Las **Web APIs** (fetch, timers, DOM, canvas...) son interfaces del navegador que realizan operaciones asíncronas fuera del hilo principal, reprogramando sus callbacks a través del event loop. Operaciones costosas en el hilo principal bloquean la interfaz.

Mecanismos de almacenamiento

Cookies

Datos asociados a dominios, enviados en cabeceras HTTP. Usadas para sesión, preferencias y seguimiento. Implicaciones de seguridad y privacidad.

localStorage

Almacenamiento clave-valor persistente por origen hasta limpieza explícita. Mayor capacidad que cookies, sin envío automático.

sessionStorage

Igual que localStorage, pero solo persiste durante la sesión de pestaña o ventana activa.

Caché

Navegadores y CDNs reutilizan respuestas y recursos estáticos. Las cabeceras Cache-Control y ETag controlan este comportamiento.

HTML vs DOM. Ejercicio

Crear el archivo index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Diferencia entre HTML y DOM</title>
</head>
<body>
  <h1 id="titulo">Hola mundo</h1>
  <p id="parrafo">Este texto viene del HTML original.</p>
  <button id="boton">Cambiar contenido</button>

  <script>
    const titulo = document.getElementById("titulo");
    const parrafo = document.getElementById("parrafo");
    const boton = document.getElementById("boton");

    console.log("Contenido inicial del h1:", titulo.textContent);

    boton.addEventListener("click", () => {
      titulo.textContent = "Ahora el DOM fue modificado";
      parrafo.textContent = "Este texto ya no es exactamente el del HTML original.";
      document.body.style.backgroundColor = "lightyellow";
    });
  </script>
</body>
</html>
```

HTML vs DOM. Ejercicio

1. Abre el archivo en el navegador el archivo anterior.
2. Haz clic derecho sobre la página y elige:
 - a. **Ver código fuente de la página**
 - b. Ahí verás el **HTML original**, que sigue diciendo:
3. Abre las herramientas de desarrollador. Presiona: F12. Ve a la pestaña **Elements** o **Elementos**. Ahí estás viendo el **DOM actual** que el navegador tiene en ese momento.
4. Haz clic en el botón cambiar contenido.
5. Observa qué cambió. Luego, responde las siguientes preguntas:
 - a. ¿Qué texto aparece inicialmente en el h1 y en el p?
 - b. ¿Qué texto aparece después de hacer clic en el botón?
 - c. ¿Qué diferencia observaste entre la opción Ver código fuente y la pestaña Elements?
 - d. ¿Cuál de los dos representa el documento original y cuál representa la estructura actual manipulada por el navegador?
 - e. ¿JavaScript modifica el DOM o el HTML original?



¡Gracias! ..❖